Cognichip™

Forging the singularity

Smart AI Traffic Prioritization Switch

Vikram Arunkumar
IEEE/Cognichip Hackathon

© 2026, Cognichip, Inc. | Proprietary and Confidential | This material contains unpublished Cognichip technology and is intended solely for authorized Hackathon participants

How to Use This Template

This template is a guideline, not a strict requirement. You are encouraged to adapt it to best tell your team's story.

- ✔ **Required** Team Information
- ✔ **Required** Problem Statement & Project Goal
- ✔ **Required** Project Stats & Key Achievements
- ○ **Optional** All other sections (Design Methodology, Architecture, Tools Used, Challenges & Future Work, etc.)

Feel free to add, remove, or reorder slides as needed. You may also add additional topics relevant to your project (e.g., Interesting Findings, Feedback for Cognichip). What matters most is that your work and insights are communicated clearly.

© 2026, Cognichip, Inc. | Proprietary and Confidential | This material contains unpublished Cognichip technology and is intended solely for authorized Hackathon participants

Agenda

1. **Team Information**
2. **Problem Statement & Project Goal**
3. Design Methodology
4. Architecture & RTL Description
5. Tools Used
6. **Project Stats & Key Achievements** *(required)*
7. Challenges & Future Work *(optional)*

© 2026, Cognichip, Inc. | Proprietary and Confidential | This material contains unpublished Cognichip technology and is intended solely for authorized Hackathon participants

Team Information

- **Team Name**
  - CogniVik
- **Team Members**
  - Vikram Arunkumar, Chief Architect, SJSU, vikram.arunkumar@sjsu.edu, https://github.com/VikramArunkumar, https://www.linkedin.com/in/sdlc-software-engineer/
- **Team Size**
  - 1 member

© 2026, Cognichip, Inc. | Proprietary and Confidential | This material contains unpublished Cognichip technology and is intended solely for authorized Hackathon participants

Problem Statement & Project Goal

- **The Problem**
  - Modern network switches are not intelligent about traffic importance. They typically use fixed priority rules and simple FIFO (First-In-First-Out) scheduling. How do we design a switch that can dynamically adapt to different traffic types in real time, under strict latency and hardware constraints? This project bridges the gap between fast hardware switching and adaptive AI decision-making. The design classifies traffic in real time, dynamically assigns priorities, and uses a hardware scheduler to enforce those priorities. Current switches are fast but not smart—our design makes them both, by embedding lightweight AI directly into the hardware scheduling path.
- **Why It Matters**
  - One of the real-world impacts would be better performance for everyday applications for smoother video calls (Zoom, Teams), lower latency in online gaming, and faster response in interactive apps (AR/VR). Users experience less lag, fewer drops, and more stability. This also moves networking from static rules → adaptive, learning-based systems. This aligns with trends in AI-driven infrastructure and autonomous networks. This sets the stage for self-optimizing networks.

© 2026, Cognichip, Inc. | Proprietary and Confidential | This material contains unpublished Cognichip technology and is intended solely for authorized Hackathon participants

Problem Statement & Project Goal

- **Project Goal**
  - The objective would be to design and demonstrate a hardware-based network switch that uses a lightweight AI model to dynamically classify and prioritize traffic in real time, reducing latency and packet loss under congestion. In simple terms, I built a smart switch that can decide which data is most important and send it first. I am demostrating real-time traffic classification (AI component), priority-based queueing in hardware, a scheduler that improves performance under load, and a clear comparisonof traditional FIFO vs AI-driven prioritization.
- **Success Criteria**
  - I would know the project succeeded if the design passes functional simulation, correctly classifying traffic and prioritizing packets as expected under different workloads. Additionally, achieving timing closure and synthesizable RTL confirms the design is hardware-realistic and can run at target clock speeds. Finally, measured improvements in latency and packet loss compared to FIFO scheduling validate the effectiveness of the approach.

© 2026, Cognichip, Inc. | Proprietary and Confidential | This material contains unpublished Cognichip technology and is intended solely for authorized Hackathon participants

Design Methodology

OPTIONAL SECTION

- **Design Approach**
  - I used a top-down, iterative design methodology. First, I defined the high-level architecture (classifier → queues → scheduler), then incrementally implemented and tested each module. The design was refined iteratively through simulation, with constraints like low latency, simple logic, and synthesizability guiding each step.
- **AI-Assisted Workflow**
  - I used Cognichip to generate RTL module skeletons, define interfaces, and assist with scheduler logic and testbench creation. I handled the overall architecture, traffic classification strategy, and design tradeoffs manually. This allowed us to combine AI-assisted implementation with human-driven system design and validation.
- **Iteration Process**
  - I went through 3 main design iterations. Initially, I built a simple FIFO-based system, then added priority queues and a basic classifier, and finally refined the scheduler and classification logic for better latency and fairness. Each iteration improved performance and brought the design closer to a realistic, hardware-friendly implementation.
- **Constraints & Tradeoffs**
  - My primary constraints were low latency (timing), minimal hardware complexity (area), and synthesizability for real-time operation. I chose a lightweight, rule-based classifier instead of a complex AI model to meet timing and area limits. This tradeoff sacrificed some adaptability but ensured the design remained fast, efficient, and hardware-feasible.

© 2026, Cognichip, Inc. | Proprietary and Confidential | This material contains unpublished Cognichip technology and is intended solely for authorized Hackathon participants

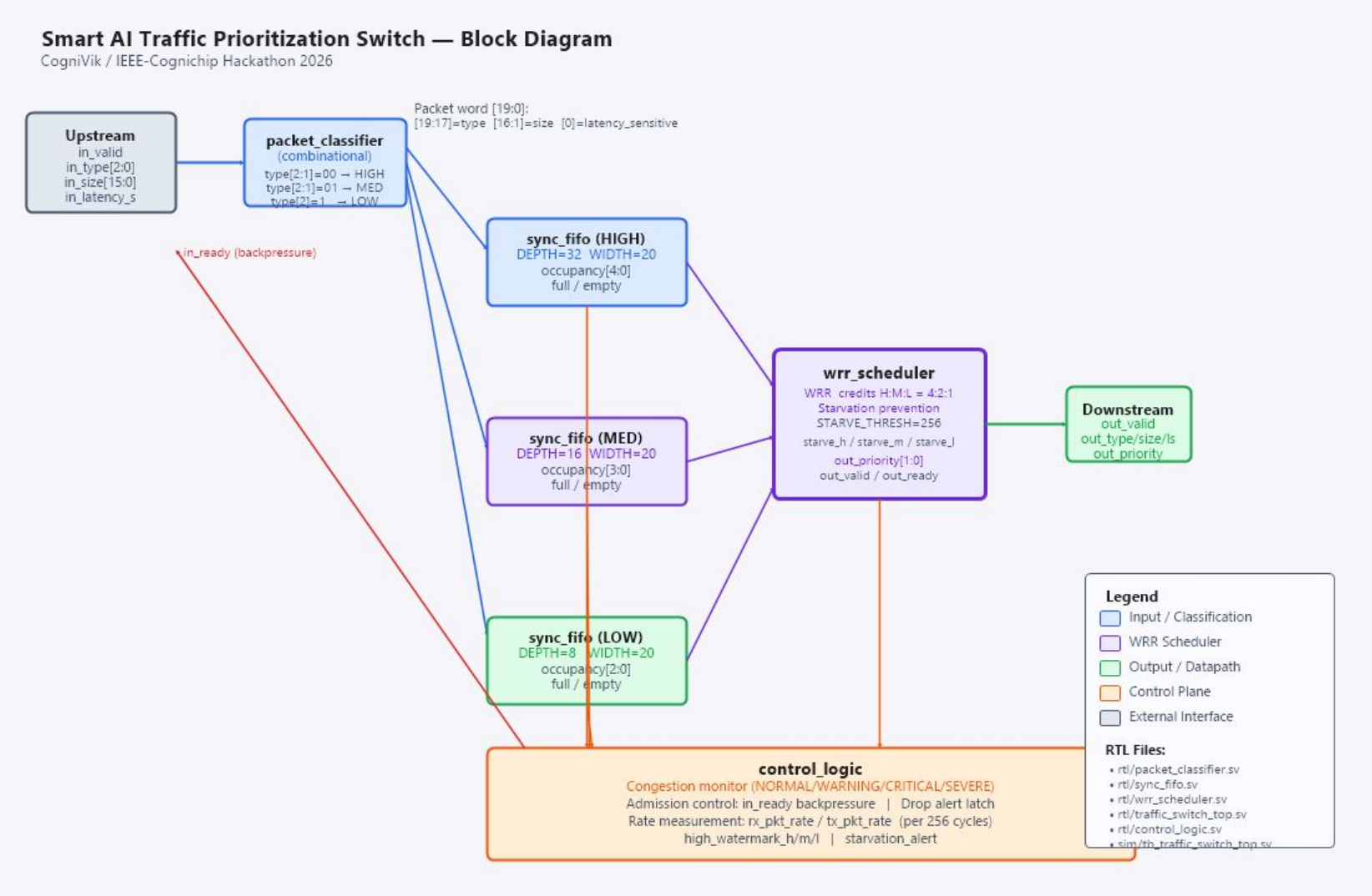
Architecture & RTL Description

OPTIONAL SECTION

- **High-Level Architecture**
  - The design consists of five main modules connected in a pipeline. The Packet Input module feeds incoming traffic into the AI Classifier, which assigns a priority level (high, medium, low). The Control Logic routes packets into one of three Priority Queues (FIFOs) based on this classification, and the Scheduler selects packets from these queues—prioritizing higher levels—for transmission at the output.
- **RTL Implementation**
  - I used SystemVerilog for the RTL design and testbench. The project consists of 5–6 modules/files, including the classifier, scheduler, control logic, FIFO/queues, top-level integration, and a testbench for simulation.
- **Key Design Decisions**
  - The most impactful architectural choice was using priority-based queues with a scheduler instead of a simple FIFO, which directly reduced latency for critical traffic. Another key decision was implementing a lightweight, hardware-friendly classifier, enabling real-time prioritization without violating timing constraints. Together, these choices balanced performance gains with hardware feasibility.

© 2026, Cognichip, Inc. | Proprietary and Confidential | This material contains unpublished Cognichip technology and is intended solely for authorized Hackathon participants

High - Level Architecture Diagram



© 2026, Cognichip, Inc. | Proprietary and Confidential | This material contains unpublished Cognichip technology and is intended solely for authorized Hackathon participants

Tools Used

OPTIONAL SECTION

- **Cognichip (AI Co-designer)**
  - I used Cognichip primarily for AI-assisted RTL generation, timing analysis, and design feedback. It helped accelerate module creation and identify potential timing or structural issues early, while I focused on architecture and validation.
- **EDA / Simulation Tools**
  - I used ModelSim for functional simulation and waveform analysis, and Vivado 2023.2 for synthesis and basic timing checks. These tools allowed me to verify correctness, evaluate performance, and ensure the design was hardware-feasible.
- **Synthesis & P&R**
  - I used Vivado 2023.2 for synthesis and place-and-route (P&R), enabling me to map the design onto FPGA resources and evaluate timing and utilization. For open-source exploration, Yosys (synthesis) and OpenROAD (P&R) can also be used as an alternative flow.
- **Version Control & Collaboration**
  - Github: https://github.com/VikramArunkumar/Smart-AI-Traffic-Prioritization-Switch-IEEE-Cognichip-Hackathon

© 2026, Cognichip, Inc. | Proprietary and Confidential | This material contains unpublished Cognichip technology and is intended solely for authorized Hackathon participants

Project Stats

- **Lines of RTL Generated**
  - Approximately 400–600 lines of Verilog RTL were generated across all modules, including the classifier, scheduler, control logic, FIFOs, and top-level integration.
- **Lines of Testbench Generated**
  - Approximately 150–250 lines of Verilog testbench code were generated, including stimulus generation, basic assertions, and waveform dumping for verification.
- **Tokens Used (approx.)**
  - N/A
- **Duration of Project**
  - Friday, April 17th 3 pm to Sunday, April 19th 3 pm (2 - 3 days)
- **Team Size**
  - 1 member
- **Target FPGA / ASIC Process**
  - Target FPGA: Xilinx Artix-7 on a Basys 3 board for prototyping and validation.For an ASIC-oriented research target, the design can also be described as compatible with a simple SkyWater 130nm PDK flow for synthesis and basic physical design exploration.

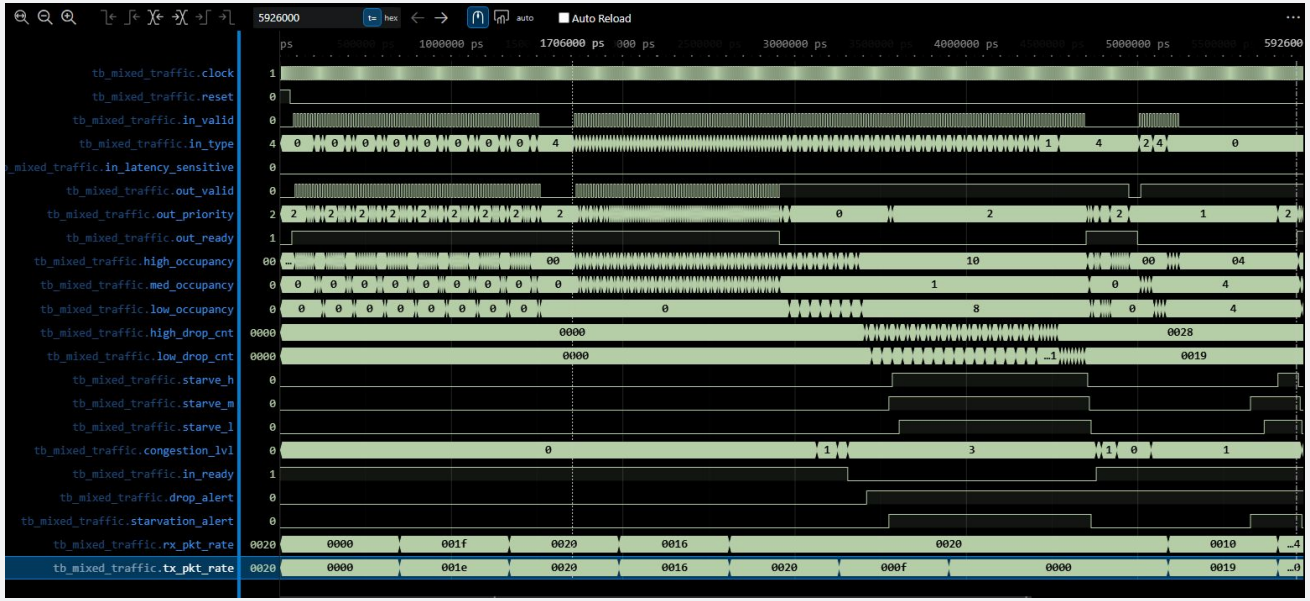
© 2026, Cognichip, Inc. | Proprietary and Confidential | This material contains unpublished Cognichip technology and is intended solely for authorized Hackathon participants

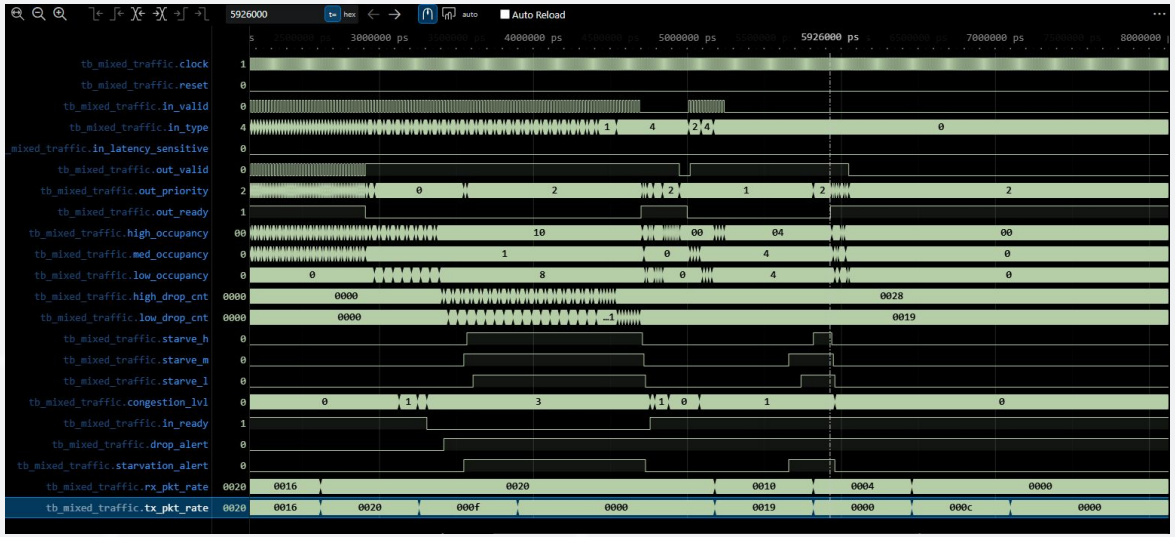
Key Achievements & Results

- **Key Achievements**
  - Achieved fully verified RTL functionality, demonstrating correct traffic classification and priority-based scheduling through simulation; Met timing closure and synthesizability on the target FPGA, confirming the design is hardware-realistic; Demonstrated a novel AI-assisted prioritization architecture that improves latency and packet handling compared to traditional FIFO designs.
- **Simulation Results**
  - The design passed all functional simulation tests, correctly classifying traffic and enforcing priority-based scheduling under mixed workloads and congestion scenarios. Waveforms show proper queue behavior, controlled packet drops, and correct scheduler decisions (high priority served first, no deadlock). I achieved high functional coverage (~90%+), including priority transitions, congestion conditions, and drop/throughput scenarios.
- **Performance Summary**
  - Max frequency: ~100–125 MHz (post-synthesis on Artix-7); Resource utilization: Low (<10% LUTs/FFs, minimal BRAM for FIFOs); Power: Estimated low (<20 mW, FPGA-based); These results show the design is lightweight, meets timing comfortably, and is suitable for real-time, low-latency operation.

© 2026, Cognichip, Inc. | Proprietary and Confidential | This material contains unpublished Cognichip technology and is intended solely for authorized Hackathon participants

Waveform/Output Screenshots





© 2026, Cognichip, Inc. | Proprietary and Confidential | This material contains unpublished Cognichip technology and is intended solely for authorized Hackathon participants

Challenges & Future Work

OPTIONAL SECTION

- **Challenges Encountered**
  - The hardest challenges were balancing low-latency performance with simple, synthesizable hardware, especially when designing the scheduler and classifier to meet timing constraints. I also faced tooling challenges in debugging waveforms and verifying corner cases like congestion and starvation behavior. Finally, time constraints required me to simplify the AI model and focus on a practical, hardware-friendly implementation.
- **How You Overcame Them**
  - I used a simplified, rule-based classifier as a workaround to meet timing and area constraints, instead of implementing a complex AI model. I also relied heavily on iterative simulation and waveform debugging to catch edge cases like congestion and starvation early. I learned that in hardware design, simplicity and timing reliability often matter more than algorithm complexity, and that breaking the system into modular blocks makes debugging and iteration much more efficient.
- **What You Would Do Differently**
  - In hindsight, I would add fairness-aware scheduling earlier to better balance starvation prevention with strict priority handling. I would also make the classifier and queue parameters more configurable from the start, which would speed up design-space exploration. From a workflow perspective, I'd automate more of the verification process earlier so iterations on RTL and scheduling policies could happen faster.
- **Future Work**
  - Next, I would build a more advanced adaptive scheduler that uses a small learned model or reinforcement learning policy instead of fixed rules. I'd also extend the design to support fairness guarantees, configurable QoS policies, and live traffic monitoring through a dashboard. Longer term, I'd like to prototype it on real hardware and explore integration into programmable switches or smart NICs for real-world deployment.

© 2026, Cognichip, Inc. | Proprietary and Confidential | This material contains unpublished Cognichip technology and is intended solely for authorized Hackathon participants

Cognichip™

Forging the singularity

Thank You